

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**  
**KHOA CÔNG NGHỆ THÔNG TIN 1**

---



**BÁO CÁO TUẦN 3**  
**MÔN THỰC TẬP CƠ SỞ**

**Đề tài: Nghiên cứu, đánh giá và triển khai  
mô hình Deep Learning tối ưu cho hệ thống phân loại rác thải**

|                              |                           |
|------------------------------|---------------------------|
| <b>Giảng viên hướng dẫn:</b> | <b>TS. Kim Ngọc Bách</b>  |
| <b>Sinh viên thực hiện:</b>  | <b>Nguyễn Văn Gia Bảo</b> |
| <b>Mã sinh viên:</b>         | <b>B23DCCN079</b>         |

*Hà Nội - 2026*

# MỤC LỤC

|                                                          |   |
|----------------------------------------------------------|---|
| 1. Mục tiêu công việc trong tuần .....                   | 3 |
| 2. Nội dung công việc đã thực hiện .....                 | 3 |
| 2.1 Tìm hiểu kỹ thuật Fine- tuning và Freeze layer ..... | 3 |
| 2.2 Tìm hiểu Training Loop .....                         | 3 |
| 2.3 Tìm hiểu các Metric đánh giá .....                   | 4 |
| 2.4 Xây dựng và huấn luyện mô hình VGG16 .....           | 4 |
| 3. Kết quả đạt được .....                                | 5 |
| 4. Khó khăn gặp phải .....                               | 6 |
| 5. Kế hoạch tuần tiếp theo .....                         | 6 |
| TÀI LIỆU THAM KHẢO .....                                 | 6 |

## 1. Mục tiêu công việc trong tuần

- Tìm hiểu kỹ thuật fine- tuning và freeze layer trong Transfer Learning
- Tìm hiểu training loop trong PyTorch: forward pass, backward pass, optimizer
- Tìm hiểu các metric đánh giá: accuracy, F1- score, confusion matrix
- Xây dựng và huấn luyện mô hình VGG16 trên dataset TrashNet
- Đánh giá kết quả mô hình trên tập test

## 2. Nội dung công việc đã thực hiện

### 2.1 Tìm hiểu kỹ thuật Fine- tuning và Freeze layer

Fine- tuning là kỹ thuật điều chỉnh mô hình pretrained cho phù hợp với bài toán mới [1]. Thay vì huấn luyện toàn bộ mô hình từ đầu, fine- tuning cho phép tái sử dụng các đặc trưng đã học từ ImageNet và chỉ điều chỉnh các lớp cuối cho phù hợp với bài toán phân loại 6 loại rác thải.

Kỹ thuật freeze layer được áp dụng để đóng băng các lớp Conv2d trong phần features của VGG16 - giữ nguyên kiến thức đã học từ ImageNet - và chỉ huấn luyện lại lớp cuối (classifier) được thay từ 1.000 class thành 6 class. Trong PyTorch, freeze layer được thực hiện bằng cách đặt `requires_grad = False` cho các tham số cần đóng băng [2].

VGG16 sử dụng kiến trúc CNN gồm các thành phần chính: Conv2d (lớp tích chập) trích xuất đặc trưng từ ảnh thông qua các bộ lọc, ReLU (hàm kích hoạt) loại bỏ các giá trị âm để tăng tính phi tuyến cho mạng, MaxPool2d giảm kích thước feature map bằng cách lấy giá trị lớn nhất trong mỗi vùng, và Linear (lớp fully connected) kết hợp toàn bộ đặc trưng để đưa ra dự đoán cuối cùng [2].

### 2.2 Tìm hiểu Training Loop

Training loop là vòng lặp huấn luyện model qua nhiều epoch. Mỗi epoch gồm 4 bước chính [2]:

- **Forward pass:** Đưa batch ảnh vào model → nhận kết quả dự đoán
- **Tính loss:** So sánh kết quả dự đoán với nhãn đúng bằng hàm `CrossEntropyLoss`
- **Backward pass:** Tính gradient bằng thuật toán backpropagation
- **Cập nhật trọng số:** Optimizer Adam điều chỉnh trọng số theo gradient

Ngoài ra, StepLR scheduler được sử dụng để tự động giảm learning rate 10 lần sau mỗi 3 epoch - giúp model học nhanh ban đầu và tinh chỉnh chính xác hơn ở giai đoạn sau.

## 2.3 Tìm hiểu các Metric đánh giá

Em đã tìm hiểu 3 metric đánh giá chính [3]:

- **Accuracy:** Tỷ lệ dự đoán đúng trên tổng số ảnh
- **F1- score:** Trung bình điều hòa giữa Precision và Recall - đánh giá chính xác hơn khi dataset mất cân bằng
- **Precision** là tỷ lệ dự đoán đúng trong tổng số lần model dự đoán một class - đo lường độ chính xác khi model khẳng định một ảnh thuộc class nào đó. **Recall** là tỷ lệ model tìm được đúng trong tổng số ảnh thật của một class - đo lường khả năng tìm ra hết các ảnh thuộc class đó. F1- score là trung bình điều hòa của Precision và Recall, cho phép đánh giá tổng thể cả 2 tiêu chí cùng lúc [3].
- **Confusion matrix:** Ma trận thể hiện mức độ nhầm lẫn giữa các class

## 2.4 Xây dựng và huấn luyện mô hình VGG16

Em đã xây dựng 3 file code chính:

**models/vgg16.py:** Load VGG16 pretrained từ torchvision, freeze phần features, thay lớp cuối từ 1.000 class thành 6 class.

**train.py:** Viết training loop hoàn chỉnh với optimizer Adam (learning rate=0.001), StepLR scheduler (step\_size=3, gamma=0.1), huấn luyện 10 epoch và tự động lưu model tốt nhất.

**evaluate.py:** Load model đã train, đánh giá trên tập test, tính accuracy, in classification report và vẽ confusion matrix.

### Cấu hình huấn luyện:

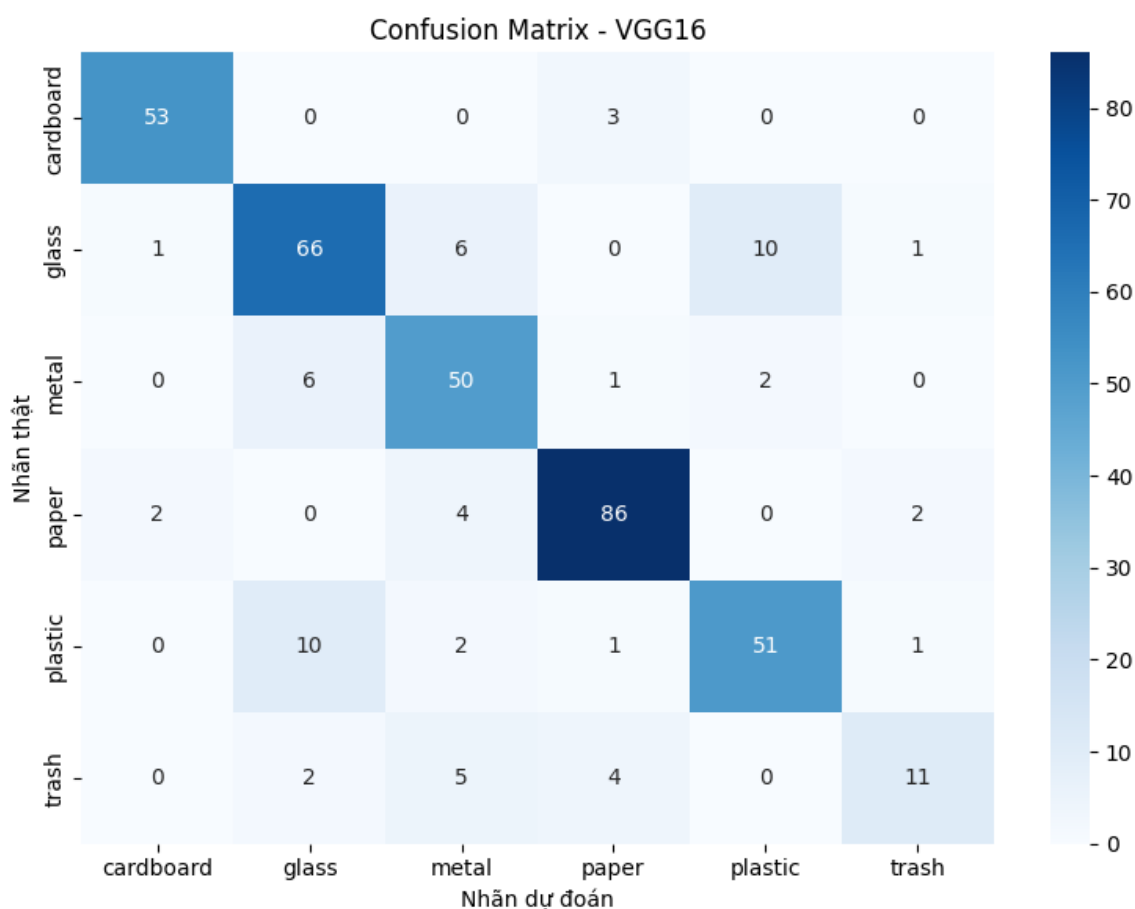
- Optimizer: Adam, learning rate = 0.001
- Scheduler: StepLR, step\_size = 3, gamma = 0.1
- Số epoch: 10
- Batch size: 32

### 3. Kết quả đạt được

- Huấn luyện thành công mô hình VGG16 trên dataset TrashNet
- Test Accuracy: 83.42%
- Macro F1- score: 0.80

Kết quả chi tiết từng class:

| Class     | Precision | Recall | F1- score |
|-----------|-----------|--------|-----------|
| Cardboard | 0.95      | 0.95   | 0.95      |
| Paper     | 0.91      | 0.91   | 0.91      |
| Plastic   | 0.81      | 0.78   | 0.80      |
| Metal     | 0.75      | 0.85   | 0.79      |
| Glass     | 0.79      | 0.79   | 0.79      |
| Trash     | 0.73      | 0.50   | 0.59      |



Class **trash** có F1- score thấp nhất (0.59) do đây là class rác hỗn hợp - trông giống nhiều class khác nên model hay nhầm lẫn.

## 4. Khó khăn gặp phải

- Các kiến thức mới trong tuần như fine-tuning, freeze layer, training loop, backpropagation, optimizer còn khá xa lạ và chưa có nền tảng trước đó nên cần nhiều thời gian tìm hiểu và tiếp thu
- Chưa quen với cách PyTorch tổ chức training loop - đặc biệt là thứ tự `zero_grad()`, `backward()`, `optimizer.step()` và lý do tại sao phải thực hiện đúng thứ tự đó
- Class trash có F1-score thấp nhất (0.59) do tính chất đặc thù là rác hỗn hợp, trông giống nhiều class khác

## 5. Kế hoạch tuần tiếp theo

- Xây dựng và huấn luyện mô hình ResNet50 trên dataset TrashNet
- Xây dựng và huấn luyện mô hình EfficientNet trên dataset TrashNet
- Đánh giá kết quả cả 2 mô hình
- Bắt đầu lập bảng so sánh 3 mô hình

## TÀI LIỆU THAM KHẢO

- [1] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345-1359, 2010.
- [2] A. Paszke et al., "PyTorch: An imperative style, high- performance deep learning library," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [3] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861-874, 2006.